LESSON 05 · CHEAT SHEET

Optional, Done Right

Java 8 → Java 21 · results use the shared course dataset
course "today" = 2026-11-07

Key rules

1. **Optional = a box with 0 or 1 value**, designed as a **return type** for "maybe no result". 8
2. **Not for** fields, parameters, collections, or replacing every null.
3. `of(x)` needs non-null (else NPE) · `ofNullable(x)` adapts null · `empty()`.
4. **orElse(x) is eager**: x is evaluated even when a value is present. **orElseGet(supplier) is lazy.** Constants → orElse; computations → orElseGet.
5. **map** transforms · **filter** keeps if true · **flatMap** when the function returns Optional (else Optional<Optional>).
6. Never `get()` unchecked; never `isPresent()` + `get()`; never `return null` from an Optional method.
7. Empty collection, not `Optional<List>`.

Canonical snippets

1 · Chain lookups (no null checks)

```
return findCustomer(id)
    .flatMap(c -> findAccount(c.id(), type))
    .flatMap(a -> latestTxn(a.number()))
    .map(Transaction::amount);
// C001 -> Optional[447.37] · C999 -> empty
```

2 · Eager vs lazy default

```
.orElse("UNKNOWN")                  // constant: fine
.orElseGet(() -> loadFromDb(id))    // only if empty
.orElseThrow(() -> new IllegalArgumentException(
        "Unknown customer " + id))
```

3 · Java 9: or + stream

```
latestOn(id, SAVINGS).or(() -> latestOn(id, CURRENT));
numbers.stream().map(Repo::findPolicy)
       .flatMap(Optional::stream)    // drops empties
       .toList();
```

API

Method	Returns	Use	Since
`of / ofNullable / empty`	Optional<T>	create	8
`map(Function)`	Optional<R>	transform	8
`flatMap(Function→Optional)`	Optional<R>	chain lookups	8
`filter(Predicate)`	Optional<T>	keep if true	8
`orElse(T)`	T	eager default	8
`orElseGet(Supplier)`	T	lazy default	8
`orElseThrow(Supplier)`	T	domain exception	8
`ifPresent(Consumer)`	void	act if value	8
`or(Supplier<Optional>)`	Optional<T>	fallback source	9
`ifPresentOrElse(c, r)`	void	both branches	9
`stream()`	Stream<T>	0 or 1 element	9
`orElseThrow()`	T	honest get()	10
`isEmpty()`	boolean	!isPresent()	11
`OptionalInt/Long/Double`	getAsInt…	no boxing, no map	8

Anti-patterns → fixes

Don't	Do
`isPresent() + get()`	`map(...).orElse(...)`
`get()` unchecked	`orElseThrow(() -> new X(...))`
`Optional` field / parameter	nullable field + Optional getter
`Optional<List<T>>`	empty `List`
`return null;`	`return Optional.empty();`
`Optional.of(maybeNull)`	`Optional.ofNullable(...)`
`orElse(expensive())`	`orElseGet(() -> expensive())`

Mini diagram: the chain



Any step empty → the rest is skipped → `Optional.empty`

Real results from the course code

Call (theory/OrElseVsOrElseGet)	Result	default called?
`orElse(C001)`	Alice Tan	yes (wasted)
`orElseGet(C001)`	Alice Tan	no
`orElse(C999)`	Walk-in customer	yes
`orElseGet(C999)`	Walk-in customer	yes

latestAmount(customer, type)	Result
`C001, SAVINGS`	`Optional[447.37]`
`C003, FIXED_DEPOSIT`	empty: account has no transactions
`C005, CURRENT`	empty: no such account
`C999, SAVINGS`	empty: no such customer

Primitive optionals: `mapToInt(seatsLeft).max()` → `OptionalInt[63]` · average load factor → `84.1%` · empty list `max()` → `OptionalInt.empty`, `orElse(0)` → 0. **Renewals within 30 days of 2026-11-07:** POL-502, POL-505, POL-508, POL-509.

Lesson 05 · sticky notes

Cut out, stick on your wall, and read one every morning.

Optional = return type

"maybe no result".
Not fields, not parameters, not lists.

orElse is EAGER

`orElse(x)` : x always runs.
`orElseGet(() -> x)` : only if empty.

flatMap for lookups

Function returns Optional?
map → Optional<Optional>. Use flatMap.

No get()

Throws "No value present".
Use orElse / orElseGet / orElseThrow.

Never return null

from an Optional method.
`return Optional.empty();`

Java 9 · 10 · 11

9: or, ifPresentOrElse, stream
10: orElseThrow()
11: isEmpty()

of vs ofNullable

`of(null)` → NPE now.
Legacy null API? `ofNullable`.

Empty list, not Optional<List>

An empty collection already means "none".